



Processou durante 6m 27s

Fiz uma análise “clínica” da versão **r0051 da Estação PEA**: estrutura, JavaScript, persistência, segurança, integridade dos registos, lógica operacional, IA, funcionamento sem rede, acessibilidade e manutenção. O ficheiro tem cerca de **8 000 linhas** e concentra toda a aplicação num único HTML.

CSREPCDouro_r0051_202608291514_...

A conclusão principal é esta: **o projecto está bastante acima de um simples protótipo de formulário e revela uma modelação operacional muito cuidada, mas eu não o colocaria ainda em exploração operacional como sistema de registo autoritativo**. Há cinco pontos que considero bloqueantes: **XSS, integridade documental, sequência de aprovação do PEA, validação de GDH e controlo da informação enviada para serviços externos/IA**.

Área	Avaliação
Modelação funcional/operacional	8,5/10
Arquitectura do estado e migrações	8/10
Interface e ergonomia	8/10
Resiliência a falhas de rede	7/10
Acessibilidade	6,5/10
Manutenibilidade	5,5/10
Segurança aplicacional	4/10
Integridade/auditabilidade documental	3/10
Prontidão para utilização operacional oficial	4,5/10

1. Problema mais grave: vulnerabilidade XSS

Há aqui uma vulnerabilidade real.

A função de escape é:

```
const esc = s => String(s??'').replace(/[<>&]/g, ...)
```

Portanto, protege `<`, `>` e `&`, mas **não protege aspas duplas nem simples**.

```
CSREPCDouro_r0051_202608291514_...
```

Isso seria aceitável para texto colocado entre elementos HTML, mas a aplicação usa a mesma função dentro de atributos HTML criados por `innerHTML`. Por exemplo:

```
value="${esc(x.cmd)}"
```

```
value="${esc(x.adj||'')}"
```

```
value="${esc(x.ct)}"
```

```
CSREPCDouro_r0051_202608291514_...
```

O mesmo padrão aparece, por exemplo, na passagem de turno.

```
CSREPCDouro_r0051_202608291514_...
```

Uma aspa introduzida num valor — manualmente ou através de um JSON importado — consegue terminar o atributo `value` e introduzir novos atributos HTML. Isto abre a porta a **stored XSS**.

O impacto aqui é particularmente sério porque a aplicação guarda ocorrências, contactos, estrutura do PCO, evolução, PEA, histórico e outros dados no armazenamento do browser. Um XSS executado nesse contexto pode manipular ou extrair esses dados.

Isto é **P0** — corrigir antes de qualquer utilização real.

A solução correcta não é criar apenas um `escAttr()`. É deixar de gerar campos contendo dados através de `innerHTML`. Para dados variáveis devem ser usados `createElement()`, `.textContent`, `.value` e propriedades DOM. Depois disso, se a aplicação for servida por HTTP/HTTPS, deve ser possível impor uma Content Security Policy razoavelmente restritiva.

2. A aplicação fala em “prova documental”, mas tecnicamente ainda não a fornece

O próprio código declara:

“Uma ocorrência gravada é prova documental de decisões de comando”

e mantém um sistema bastante bom de migrações do estado, actualmente na versão 10.

```
CSREPCDouro_r0051_202608291514_...
```

Mas tecnicamente o estado é simplesmente serializado como JSON e armazenado através de `localStorage`, `window.storage` ou, em último recurso, memória da sessão.

CSREPCDouro_r0051_202608291514_...

A exportação é igualmente um JSON normal:

```
{tipo:"peaapp:ocorrencia", versao:..., estado:0}
```

e a importação verifica essencialmente se é JSON, se é um objecto e se contém `meta`, seguindo depois para as migrações.

CSREPCDouro_r0051_202608291514_...

Não existe assinatura digital, MAC, hash encadeado ou outro mecanismo de integridade criptográfica.

Existe uma função chamada `assinaturaDispositivo()`, mas tecnicamente ela é apenas uma concatenação de contagens:

setores | meios | aéreos | funções | número da ocorrência

Não é uma assinatura criptográfica.

CSREPCDouro_r0051_202608291514_...

E o próprio código reconhece correctamente que o encerramento:

“Não é selo criptográfico”

e que a ocorrência pode ser reaberta.

CSREPCDouro_r0051_202608291514_...

Portanto, neste momento o sistema oferece um **registo operacional local**, não “prova documental” no sentido forte de integridade, autenticidade ou não repúdio.

Uma pessoa com acesso ao computador pode editar um JSON exportado, alterar um GDH, apagar uma evolução, substituir o nome de quem determinou alguma coisa e voltar a importar o ficheiro. A aplicação não tem forma de detectar a alteração.

Para uso institucional, eu alteraria até a terminologia enquanto isto não for resolvido.

3. Detectei um erro concreto no tratamento dos GDH

A função `parseGDH()` parece validar o formato:

DDHHMMMMAA

mas depois constrói directamente um `new Date(...)` e testa apenas `isNaN()`.

CSREPCDouro_r0051_202608291514_...

O problema é que o JavaScript **normaliza datas impossíveis**.

Testei exactamente a lógica deste código:

- `311000FEV26` é aceite e convertido em **03/03/2026 10:00**;
- `321000JAN26` é aceite e convertido em **01/02/2026 10:00**.

Ou seja, um erro de introdução de dados não é recusado: **é silenciosamente transformado noutro GDH válido**.

Num sistema que usa tempos para evolução, rendições, validade do PEA, nomeações, passagem de comando e histórico, isto é grave.

Além disso, `gdhAgora()` utiliza directamente o relógio e o fuso horário do computador local. `CSREPCDouro_r0051_202608291514_...`

Se o relógio ou timezone da estação estiverem errados, os GDH também ficam errados.

Eu faria validação estrita de dia/mês/ano/hora/minuto, com round-trip, e guardaria internamente os instantes em ISO 8601 com offset ou timestamp UTC. O GDH seria apenas a representação operacional apresentada ao utilizador.

4. Há uma inconsistência importante no processo de aprovação do PEA

Este ponto merece atenção porque não é um problema meramente informático.

`emitirPEA()` faz:

1. `gerarPEA()` ;
2. imediatamente `gerarOrdens()` ;
3. constrói o PEA;
4. cria o controlo das missões;
5. adiciona-o a `O.peas` ;
6. regista “**PEA n.º X emitido**”.

`CSREPCDouro_r0051_202608291514_...`

Contudo, o prompt dado à segunda chamada de IA afirma:

“A célula de planeamento elaborou o plano estratégico de acção n.º X e o **COS aprovou-o.**”

`CSREPCDouro_r0051_202608291514_...`

Nesse momento **não ocorreu qualquer acção de aprovação pelo COS entre as duas operações**.

Isto cria um desfasamento entre o processo real e aquilo que o software declara à IA.

O botão diz “Emitir proposta de PEA”, mas internamente as ordens de missão são produzidas partindo da premissa de que a proposta já foi aprovada.

Eu considero isto outro **P0 operacional**.

O modelo deveria ser explicitamente:

Rascunho → Proposta de PEA → Aprovação/determinação pelo COS → PEA em vigor → Geração/difusão das ordens de missão → Revisão/substituição.

A aprovação deve ter, no mínimo, identidade/função, GDH e registo próprio. A IA nunca deve poder “presumir” esse acto.

5. Informação operacional é efectivamente enviada para IA

A implementação da IA está bem pensada num aspecto: os três modos são explícitos.

- ambiente Claude → `api.anthropic.com`;
- aplicação servida → `/pea/llm`;
- `file://` → funcionamento determinístico sem IA.

CSREPCDouro_r0051_202608291514_...

O problema não é existir IA; é perceber **o que sai da estação**.

O contexto enviado inclui:

- número da ocorrência;
- local;
- PCO;
- fase SGO;
- área;
- nome do ficheiro de perímetro;
- sectores e meios;
- pontos sensíveis;
- PEA anterior;
- evolução desde o último PEA;
- métricas meteorológicas.

CSREPCDouro_r0051_202608291514_...

Isto pode constituir informação operacional relevante.

Antes de utilização real, tem de existir uma decisão institucional explícita sobre classificação da informação, minimização, autorização para processamento externo, conservação, logging e modelo autorizado.

Eu colocaria ainda uma indicação antes do envio:

“Os seguintes dados vão sair deste equipamento: ...”

e permitiria trabalhar por defeito em modo determinístico.

Há ainda outros fluxos externos: geocodificação, meteorologia, IPMA, Photon/OpenStreetMap e Overpass. A implementação destes últimos é, aliás, resiliente e tenta vários servidores.

CSREPCDouro_r0051_202608291514_...

6. Persistência: boa concepção de contingência, armazenamento insuficiente para produção

Gostei bastante da estratégia de fallback:

`window.storage` → `localStorage` → memória da sessão.

E, quando só existe memória da sessão, o sistema apresenta deliberadamente um aviso permanente de que tudo se perderá ao fechar e recomenda a exportação.

CSREPCDouro_r0051_202608291514_...

Isto é boa engenharia defensiva.

Mas `localStorage` continua a não ser o local onde eu confiaria a operação de um PCO.

O estado completo é escrito numa chave e o índice noutra, sem transacção conjunta.

CSREPCDouro_r0051_202608291514_...

Além disso, limpeza do browser, mudança de perfil, quota, avaria do equipamento ou erro humano podem fazer desaparecer os dados.

Para uma versão operacional escolheria IndexedDB ou uma pequena camada local de persistência transaccional, com journal de alterações e snapshots. A exportação JSON continuaria a existir como mecanismo de interoperabilidade e contingência.

7. A importação precisa de validação por esquema

Há um aspecto positivo: versões futuras são recusadas em vez de serem interpretadas à sorte e existe uma cadeia explícita de migrações. Isso é **muito bom**.

CSREPCDouro_r0051_202608291514_...

Mas a validação de uma ocorrência importada é superficial:

JSON válido → objecto → existência de `meta` → migração.

CSREPCDouro_r0051_202608291514_...

Falta validar rigorosamente tipos, limites, arrays, enumerações, comprimentos e propriedades permitidas.

Isto é simultaneamente um problema de robustez e de segurança e está directamente relacionado com o XSS identificado.

Um JSON Schema versionado resolveria boa parte do problema.

8. Rede e funcionamento degradado estão bastante bem tratados

Aqui a avaliação é positiva.

`fetchT()` distingue:

- sem rede;
- timeout;
- resposta HTTP recusada;
- falha genérica;

usa `AbortController`, timeouts e cache temporária das chamadas GET.

CSREPCDouro_r0051_202608291514_...

O uso de vários mirrors Overpass e fallback para Photon também é uma boa escolha.

CSREPCDouro_r0051_202608291514_...

E a geração do PEA possui fallback determinístico se a IA falhar.

CSREPCDouro_r0051_202608291514_...

Há, contudo, uma lacuna assumida pelo próprio projecto: **a última previsão meteorológica não fica retida como contingência persistente**. O próprio interface identifica isso como próximo passo.

CSREPCDouro_r0051_202608291514_...

Num TO, onde a conectividade é precisamente uma das coisas em que menos confiaria, eu tornaria a aplicação verdadeiramente **offline-first**: fontes incluídas localmente, assets locais, última meteorologia válida persistida, carimbo da idade dos dados e PWA/service worker se for servida por browser.

9. Interface e acessibilidade

Há bastante trabalho bem feito:

- estados de `focus-visible`;
- contraste deliberadamente tratado;
- `prefers-reduced-motion`;
- desenho responsivo;
- impressão A4 específica;

- estados que não dependem exclusivamente da cor.

CSREPCDouro_r0051_202608291514_...

Isto é melhor do que a maioria dos protótipos desta dimensão.

Mas a acessibilidade não está fechada. Há controlos gerados dinamicamente — comandante, adjunto, contacto, meios, operacionais, tipologia, origem, etc. — que visualmente se entendem pela grelha, mas não têm todos um `<label>` programaticamente associado.

CSREPCDouro_r0051_202608291514_...

Para um leitor de ecrã, “campo de texto” sem contexto não é suficiente.

Precisaria de uma passagem WCAG 2.2: `label`, `fieldset/legend`, `aria-describedby`, anúncios `aria-live` para estados operacionais e verificação completa da navegação apenas por teclado.

10. Arquitectura e manutenção

Aqui há uma mistura curiosa: **a arquitectura conceptual é boa, mas a arquitectura física já está a pedir modularização.**

O estado está claramente dividido em Comando, Planeamento, Operações, Logística, turno, encerramento, cumprimento, evolução e PEA. Há migrações explícitas e comentários surpreendentemente detalhados. O estado actual também separa correctamente, por exemplo, comunicações/logística das funções do PCO.

CSREPCDouro_r0051_202608291514_...

Por outro lado, são cerca de 8 000 linhas num único HTML, com uma quantidade muito elevada de construção dinâmica através de `innerHTML`.

Validei adicionalmente o ficheiro por análise estática:

a sintaxe JavaScript passa no `node --check` e não encontrei IDs HTML duplicados.

Portanto, não estamos perante código “partido” ou improvisado. O problema é a escalabilidade da manutenção.

Eu separaria o código em módulos de domínio, estado/persistência, integrações, renderer, conformidade, meteorologia, PEA e passagem de turno. E colocaria testes unitários precisamente nas partes em que um erro é operacionalmente perigoso: GDH, migrações, regras DON/SGO, cálculo das rendições, validade/divergência do PEA, importação e encerramento.

A ordem em que eu interviria

1. **P0 — eliminar a vulnerabilidade XSS**, retirando dados variáveis dos `innerHTML` e implementando CSP depois da refactorização.
2. **P0 — corrigir GDH e relógio**, com validação estrita e representação temporal inequívoca.
3. **P0 — criar o estado formal de aprovação pelo COS**, impedindo a geração de ordens antes dessa aprovação.
4. **P0 — assegurar integridade/auditabilidade**, com log append-only, hash encadeado/assinatura, identificação do autor e exportações verificáveis.
5. **P0 — controlar a saída de informação**, especialmente contexto enviado ao LLM e coordenadas/dados operacionais enviados a terceiros.
6. **P1 — substituir a persistência principal**, acrescentar validação JSON Schema, recuperação e cache offline persistente.
7. **P1 — modularizar, automatizar testes e fechar WCAG**, preservando a interface e a lógica operacional existentes.

A minha apreciação global é, portanto, bastante definida: **a r0051 tem uma base funcional e de modelação suficientemente boa para justificar continuar a evolui-la; não a reescreveria de raiz. Mas ainda a classificaria como protótipo operacional avançado, não como sistema de produção.** Os problemas que encontrei não exigem deitar fora o trabalho feito, mas alguns — especialmente **XSS, GDH, aprovação do COS e integridade documental** — têm mesmo de ser resolvidos antes de lhe confiar um registo operacional real.

Uma ressalva importante: nesta análise avaliei a **coerência interna do código e das regras implementadas**. Não confrontei ainda, artigo por artigo, as interpretações do **Despacho n.º 4067/2024, DON n.º 1/DIOPS e DON n.º 2/DECIR 2026** com os documentos oficiais. Essa seria a segunda análise que eu faria antes de considerar a aplicação validada operacionalmente.

Fontes

hoje 21:03



CSREPCDouro_r0057_...
Ficheiro

Análise muito boa e útil, mas entretanto houve alterações, analisa esta última versão à luz dos mesmos olhos

Processou durante 2m 55s

Sim. A r0057 está claramente melhor do que a r0051 e há alterações de substância, não apenas cosméticas. Mas há uma conclusão importante: **as melhorias incidiram sobretudo em auditabilidade, proveniência dos dados, meteorologia e UX; três dos bloqueios críticos que identifiquei na r0051 continuam praticamente intactos: XSS, GDH e aprovação formal do PEA pelo COS.**

Além da leitura integral dirigida aos pontos críticos, fiz testes estáticos ao ficheiro real: o JavaScript passa no `node --check`, não encontrei IDs HTML duplicados e testei directamente a nova implementação SHA-256 e o parser de GDH.

Comparação directa r0051 → r0057

Área	r0051	r0057	Evolução
Modelação funcional/operacional	8,5	9,0	↑
Arquitectura do estado/migrações	8,0	8,7	↑
Interface/ergonomia	8,0	8,6	↑
Resiliência de rede/offline	7,0	7,2	↗
Acessibilidade	6,5	7,1	↑
Manutenibilidade	5,5	5,8	↗
Segurança aplicacional	4,0	4,0	=
Integridade/auditabilidade	3,0	5,8	↑↑
Prontidão para produção operacional	4,5	5,5	↑, mas ainda não

A evolução é real. A diferença entre o **9/10 funcional** e o **5,5/10 de prontidão operacional** explica-se precisamente pelos problemas abaixo.

1. XSS: continua a ser P0 — não foi corrigido

A função continua exactamente com o mesmo problema:

JavaScript

```
const esc = s => String(s??'').replace(/[<>]/g, ...)
```

Ou seja, escapa `<`, `>` e `&`, mas **continua sem escapar** `"` e `'`.

CSREPCDouro_r0057_202608291830_...

E os dados continuam a ser introduzidos em atributos HTML construídos através de `innerHTML`. Por exemplo:

HTML

```
<input ... value="${esc(x.cmd)}">
<input ... value="${esc(x.adj)}">
<input ... value="${esc(x.ct)}">
```

CSREPCDouro_r0057_202608291830_...

Fiz um levantamento estático da r0057: existem **80 utilizações de** `innerHTML`, e encontrei pelo menos **cinco inserções de** `esc()` **directamente em atributos** `value`.

Há ainda uma situação mais perigosa no arquivo:

HTML

```
onclick="abrirOcc('${esc(x.num)}')"
```

CSREPCDouro_r0057_202608291830_...

Aqui o número da ocorrência entra dentro de uma string JavaScript delimitada por `'...'`, mas `esc()` **não escapa** `'`.

Confirmei isso executando exactamente a função: aspas simples e duplas passam intactas.

Portanto, continua a existir possibilidade de **stored XSS** através de dados guardados/importados.

Isto continua a ser **P0 absoluto**.

A solução que recomendei antes mantém-se: não tentar remendar `esc()`, mas eliminar dados variáveis dentro de HTML construído por concatenação. Para campos e botões, usar `createElement()`, `.value`, `.textContent`, `dataset`, `addEventListener()`, etc.

Isso eliminaria também os `onclick="..."` inline, permitindo depois introduzir uma CSP decente.

2. O erro dos GDH continua exactamente presente

O `parseGDH()` continua:

JavaScript

```
const d = new Date(2000+ +m[5], mes, +m[1], +m[2], +m[3]);  
return isNaN(d.getTime()) ? null : d;
```

CSREPCDouro_r0057_202608291830_...

Voltei a testar a implementação real da r0057.

Resultados:

Introduzido	Resultado da aplicação
311000FEV26	03/03/2026 10:00
321000JAN26	01/02/2026 10:00
292400FEV26	02/03/2026 00:00
291000FEV25	01/03/2025 10:00
291000FEV24	29/02/2024 10:00 — correcto

Portanto, continuam a ser aceites:

- 31 de Fevereiro;
- dia 32;
- hora 24;
- 29 de Fevereiro num ano não bissexto.

O JavaScript normaliza-os silenciosamente.

E existe um segundo problema: o registo de evolução nem sequer obriga a que o GDH seja interpretável. Faz simplesmente:

JavaScript

```
g: $("e-gdh").value.trim() || gdhAgora()
```

CSREPCDouro_r0057_202608291830_...

Isto significa que `ABCD`, por exemplo, pode entrar na evolução como GDH.

Para mim continua **P0**, porque os tempos alimentam POSIT, rendições, validade, sequência documental, passagem de turno, etc.

A correcção deve validar explicitamente:

```
dia + hora + minuto + mês + ano
```

e depois fazer round-trip:

```
Date → componentes == componentes originais.
```

Não basta `isNaN()`.

3. Aprovação pelo COS: continua sem estar modelada

Este é provavelmente o problema operacional mais importante que continua na r0057.

A própria interface diz correctamente:

«Proposta de PEA — a elaboração é da célula; a aprovação e determinação pertencem ao COS.»

E o botão continua a chamar-se **“Emitir proposta de PEA”**.

```
CSREPCDouro_r0057_202608291830_...
```

O prompt de Planeamento também diz correctamente:

“para aprovação pelo COS”

e diz expressamente que as ordens de missão só são transmitidas **depois de o COS aprovar o plano**.

```
CSREPCDouro_r0057_202608291830_...
```

Só que imediatamente a seguir o programa chama Operações dizendo:

“o COS aprovou-o”

e fornece ao modelo um:

```
PLANO ESTRATÉGICO DE AÇÃO APROVADO
```

```
CSREPCDouro_r0057_202608291830_...
```

E `emitirPEA()` confirma a sequência:

```
gerarPEA()  
↓  
gerarOrdens()  
↓  
criar PEA  
↓  
O.peas.push(pea)  
↓  
"PEA n.º X emitido"
```

Não existe qualquer decisão do COS no meio.

CSREPCDouro_r0057_202608291830_...

Portanto continua a haver uma **ficção de aprovação** dentro da lógica da aplicação.

E isto é ainda mais evidente porque o documento impresso contém:

Aprovado — O COS: _____

ou seja, a aprovação só vai aparentemente acontecer no papel, depois de o sistema já ter produzido as ordens.

Eu alteraria o modelo para algo como:

**PROPOSTA → APROVAÇÃO PELO COS → PEA EM VIGOR → ORDENS DE MISSÃO
→ DIFUSÃO**

E acrescentaria ao estado algo deste género:

```
estado: "proposta" | "aprovado" | "substituído"  
aprovacao:  
  gdh  
  por  
  função  
  observacao
```

Só `aprovarPEA()` poderia chamar `gerarOrdens()`.

Continua **P0**.

4. Integridade documental: aqui houve uma melhoria muito significativa

Aqui a r0057 responde directamente a uma das minhas críticas anteriores.

Foi introduzido um SHA-256 do estado com serialização canónica.

```
CSREPCDouro_r0057_202608291830_...
```

Há inclusive uma boa decisão conceptual no código:

“Não é assinatura: quem alterar o pacote pode recalculer o resumo.”

Isto está **tecnicamente correcto** e evita vender um hash como se fosse assinatura digital.

```
CSREPCDouro_r0057_202608291830_...
```

Também testei a implementação SHA-256 da aplicação contra dois vectores conhecidos:

```
SHA256("")
e3b0c44298fc1c149afb4c8996fb924...
```

e

```
SHA256("abc")
ba7816bf8f01cfea414140de5dae2223...
```

A implementação da r0057 produziu os resultados correctos.

Muito positivo.

O pacote passou também a levar:

```
app
ficheiro
g
sha
estado
```

```
CSREPCDouro_r0057_202608291830_...
```

E o encerramento ganhou um `encerramento.sha` ; isto motivou inclusivamente a passagem do estado para a versão 11.

```
CSREPCDouro_r0057_202608291830_...
```

É uma evolução substancial.

Mas convém manter a distinção:

SHA-256 = integridade/detecção de alterações.

Não fornece:

autenticidade + identidade do autor + não repúdio.

Uma pessoa que altere o JSON e recalcule o SHA continua a produzir um pacote que confere.

Para chegar ao patamar seguinte seria necessária uma assinatura ou MAC baseado numa chave que não acompanhe o próprio ficheiro.

5. Há um erro novo bastante sério na identificação da revisão

Este escapou na montagem da r0057.

O rodapé diz correctamente:

r0057 — CSREPCDouro_r0057_202608291830_EstacaoPEA_CLD.html

mas imediatamente no JavaScript aparece:

JavaScript

```
const REVISAO_APP = "r0053",  
      FICHEIRO_APP = "CSREPCDouro_r0053_202608291604_EstacaoPEA_CLD.html";
```

CSREPCDouro_r0057_202608291830_...

Isto é particularmente infeliz porque essas duas constantes foram introduzidas precisamente para melhorar a **proveniência** dos ficheiros.

Quando a r0057 exportar uma ocorrência, o pacote leva:

JavaScript

```
app: REVISAO_APP,  
ficheiro: FICHEIRO_APP
```

CSREPCDouro_r0057_202608291830_...

Ou seja:

um JSON produzido pela r0057 vai declarar que foi produzido pela r0053.

E o nome da exportação também usa `REVISAO_APP` ; logo provavelmente aparecerá `_r0053_`.

Não afecta o cálculo operacional, mas **destrói precisamente a rastreabilidade de versão que se tentou criar.**

Eu corrigiria isto imediatamente:

```
REVISAO_APP = "r0057"
```

```
FICHEIRO_APP = "CSREPCDouro_r0057_202608291830_EstacaoPEA_CLD.html"
```

Classificação: **P1 alto**.

6. O mecanismo de carimbo está bom, mas um carimbo inválido deveria exigir decisão explícita

A aplicação passou a verificar correctamente o SHA do ficheiro antes de o importar.

```
CSREPCDouro_r0057_202608291830_...
```

Isso é bom.

Mas existe uma decisão de desenho questionável: se o SHA **não conferir**, a ocorrência é importada na mesma.

O código diz expressamente:

“Avisa, não recusa.”

```
CSREPCDouro_r0057_202608291830_...
```

E de facto:

```
conferirCarimbo()  
O = estado  
persistir()  
aviso: carimbo não confere
```

```
CSREPCDouro_r0057_202608291830_...
```

Percebo o raciocínio operacional: num PCO pode ser preferível recuperar um ficheiro suspeito do que ficar sem nada.

Mas eu faria:

carimbo válido → importa normalmente

sem carimbo → importa como legado

carimbo inválido → diálogo explícito “Importar como conteúdo não verificado?”

E o estado deveria transportar:

```
integridadeImportacao: "valida" | "legado" | "falhou"
```

para que o alerta não desapareça depois de 5,5 segundos.

7. A importação da Gestão PCO está bastante mais madura do que a importação da própria ocorrência

Aqui existe uma diferença interessante.

O contrato da Gestão PCO já valida:

- envelope;
- `tipo`;
- versão inteira;
- versão máxima suportada;
- versões 1.x correctamente por componentes numéricos;
- formatos legados;
- estados conhecidos/desconhecidos.

```
CSREPCDouro_r0057_202608291830_...
```

Isto está bem pensado.

Já a importação da ocorrência continua essencialmente:

```
JSON válido?  
é objecto?  
tem meta?  
→ migrar
```

```
CSREPCDouro_r0057_202608291830_...
```

Ainda não existe validação estrutural equivalente para:

- `peas`;
- `evolucao`;
- `fita`;
- `turno`;
- `logistica`;
- `pco`;
- `tipos`;
- enumerações;

- limites;
- cumprimentos;
- campos inesperados.

Portanto, a recomendação de **JSON Schema/versionamento formal do contrato de ocorrência** continua válida.

Especialmente porque isso também reduz enormemente a superfície do problema XSS.

8. DECIR: excelente correcção conceptual

Na r0051 o calendário DECIR estava essencialmente embutido como se pudesse valer indefinidamente.

Agora existe:

JavaScript

```
const DECIR_ANOS = {  
  2026: ...  
}
```

e, se não existir uma tabela para o ano, `nivelDECIR()` devolve vazio em vez de reutilizar silenciosamente 2026. `CSREPCDouro_r0057_202608291830_...`

O comentário resume muito bem a decisão:

“um ano sem tabela publicada não se adivinha a partir do ano anterior.”

Isto é exactamente a atitude que espero de uma aplicação operacional.

É uma melhoria **muito boa**.

9. Meteorologia: também houve melhoria conceptual

Os limiares deixaram de estar dispersos/implícitos e passaram para uma configuração explícita:

```
rotação: 50°  
vento mínimo: 8 km/h  
convectivo: ≥0,2 mm/h  
janela mínima: 2 h
```

HR crítica: ≤20 %

E o próprio código declara correctamente que **não são doutrina, mas heurísticas da aplicação**.

CSREPCDouro_r0057_202608291830_...

Isto é excelente do ponto de vista de auditabilidade científica/operacional: quem audita consegue distinguir:

“a norma determina isto”

de

“o algoritmo usa esta heurística”.

Na r0051 essa fronteira era menos clara.

10. Mas a contingência meteorológica offline continua por resolver

O próprio programa continua a dizer:

“Próximo passo: reter a última previsão obtida, para que uma perda de rede não deixe o meteograma vazio.”

CSREPCDouro_r0057_202608291830_...

Portanto esse ponto da análise anterior permanece.

Num PCO eu guardaria:

```
dados meteorológicos
fonte
instante de obtenção
validade/idade
coordenadas utilizadas
```

e mostraria algo como:

Última previsão disponível — obtida há 1 h 37 min

em vez de ficar sem dados após perda de rede.

11. IA e saída de informação: praticamente inalterado

O modo continua bem declarado:

- Anthropic no ambiente Claude;

- relé local quando servido;
- determinístico em `file://`.

CSREPCDouro_r0057_202608291830_...

Mas o contexto continua a incluir informação operacional e é enviado ao modelo.

Não encontrei uma nova etapa entre “Emitir proposta” e `llm()` que permita ao utilizador rever/autorizar especificamente o conjunto de dados que vai sair.

Além disso, no modo Claude a chamada continua directamente destinada ao endpoint do modelo; no modo servido, ao relé local.

A minha recomendação anterior mantém-se: uma política institucional deve determinar o que pode sair, e a aplicação deveria indicar pelo menos:

Modo de processamento

Destino

Dados transmitidos

antes da primeira utilização de IA numa ocorrência.

E eu manteria o modo determinístico como mecanismo de contingência e, idealmente, como configuração institucional seleccionável.

12. Persistência continua a ser localStorage/storage/memória

Também não mudou estruturalmente.

A ocorrência é serializada integralmente e gravada no armazenamento disponível; o índice é gravado à parte.

CSREPCDouro_r0057_202608291830_...

Não é transaccional.

O SHA melhora a integridade dos snapshots exportados e do encerramento, mas não transforma `localStorage` numa base de dados operacional robusta.

Continuaria a recomendar IndexedDB ou uma pequena camada local transaccional/jornalizada para uma futura versão de exploração.

13. Acessibilidade: melhoria real, mas incompleta

As ajudas são agora transformadas em controlos colapsáveis com:

HTML

```
button  
aria-expanded
```

e actualização do estado expandido/recolhido.

CSREPCDouro_r0057_202608291830_...

O botão global também abre/fecha todas as ajudas e actualiza `aria-pressed`.

CSREPCDouro_r0057_202608291830_...

É uma evolução boa.

Mas o problema das linhas dinâmicas da setorização permanece: os campos “estado”, “comandante”, “adjunto”, “contacto”, “meios”, “operacionais” continuam a ser gerados sob cabeçalhos visuais, sem `<label>` individual ou `aria-label` associado a cada input.

CSREPCDouro_r0057_202608291830_...

Portanto um leitor de ecrã continua a ter uma experiência muito inferior à visual.

Também não encontrei `aria-live` para avisos dinâmicos.

A acessibilidade melhorou, mas ainda não chamaria WCAG 2.2 fechada.

14. Proveniência das coordenadas: muito boa adição

Isto é pequeno em código mas muito importante operacionalmente.

A r0057 passou a guardar:

```
meta.coordFonte
```

e distingue se a coordenada veio de:

- introdução manual;
- geocodificação;
- Gestão PCO;
- etc.

A origem acompanha agora o próprio campo quando a ocorrência muda de posto.

CSREPCDouro_r0057_202608291830_...

Isto é exactamente o tipo de metadado que uma aplicação operacional deve preservar.

15. Manutenção: está melhor organizada conceptualmente, mas continua monolítica

A r0051 tinha cerca de 8 000 linhas.

A r0057 tem **8 542 linhas** e continua a ser um único HTML.

CSREPCDouro_r0057_202608291830_...

Comparei as duas versões directamente:

- aproximadamente **665 linhas foram adicionadas/substituídas**;
- não foram removidas funções importantes;
- foram acrescentadas funções de integridade, ajuda colapsável, proveniência e DECIR;
- a estrutura conceptual está claramente mais madura.

Mas o monólito continua a aumentar.

Quanto mais cresce, maior a probabilidade de coisas como o erro **r0057** → **REVISAO_APP r0053** escaparem.

Neste momento já justificaria separar pelo menos:

```
state.js
storage.js
integrity.js
time.js
meteo.js
pco-import.js
compliance.js
pea.js
operations.js
ui.js
```

mesmo que a distribuição final continue a ser empacotada num único HTML.

Situação dos quatro P0 que identifiquei na r0051

Problema	r0051	r0057
Stored XSS	✗	✗ não resolvido

Problema	r0051	r0057
GDH inválidos	✗	✗ não resolvido
Aprovação COS antes das ordens	✗	✗ não resolvido
Integridade documental	✗	🟡 parcialmente resolvido com SHA-256

É aqui que está a questão central.

A r0057 é claramente uma aplicação melhor do que a r0051. O salto em integridade, proveniência, transparência dos algoritmos e UX é evidente.

Mas, paradoxalmente, **a avaliação de segurança não sobe**, porque a vulnerabilidade XSS original continua no mesmo sítio e a fronteira de decisão entre “proposta” e “PEA aprovado” continua sem existir.

O que eu corrigiria agora, por esta ordem

1. **P0 — XSS:** eliminar dados variáveis de `innerHTML / onclick`.
2. **P0 — GDH:** parser estrito + validação na entrada de todos os GDH.
3. **P0 — workflow COS:** criar estado formal `proposta → aprovada → em vigor`; ordens apenas depois da aprovação.
4. **P1 imediato — corrigir** `REVISAO_APP/FICHEIRO_APP` **de r0053 para r0057.**
5. **P1 — hash inválido:** exigir confirmação expressa e conservar estado “integridade não verificada”.
6. **P1 — contrato da ocorrência:** validação estrutural completa, ao nível da que já começa a existir para Gestão PCO.
7. **P1 — cache meteorológica persistente/offline.**
8. **P2 — modularização + testes automáticos + fecho WCAG.**

Há ainda uma conclusão que considero positiva: **eu continuo sem ver razão para reescrever esta aplicação de raiz.** A arquitectura conceptual está a ficar bastante boa. A prioridade agora devia deixar de ser acrescentar funcionalidades durante algumas revisões e passar para uma **fase de hardening**: segurança, tempo, estados de decisão, validação, testes e integridade.

Se fizerem isso, a aplicação pode dar um salto muito maior de r0057 para a próxima versão do que acrescentando mais dez funcionalidades.

O ChatGPT pode cometer erros. Considere verificar informações importantes.